

Hướng Dẫn Sử Dụng: Security Testing cho EShop

Tài liệu này hướng dẫn chạy quy trình kiểm thử bảo mật của nhóm 06 cho EShop, kết hợp SAST bằng Semgrep và DAST bằng OWASP ZAP. Mục tiêu là phát hiện lỗi từ hai góc nhìn khác nhau: mã nguồn và hành vi runtime của ứng dụng.

1. Giới thiệu ngắn OWASP Top 10 2025

OWASP Top 10 là danh sách các nhóm rủi ro bảo mật web phổ biến và quan trọng nhất. Trong workflow này, nhóm dùng danh sách này để chọn ruleset Semgrep và scan policy ZAP.

Mã	Nhóm rủi ro	Ý nghĩa ngắn
A01	Broken Access Control	Ứng dụng kiểm soát quyền truy cập không chặt, cho phép user truy cập dữ liệu hoặc chức năng ngoài quyền.
A02	Security Misconfiguration	Cấu hình server, framework, CORS, header hoặc môi trường triển khai không an toàn.
A03	Software Supply Chain Failures	Rủi ro từ thư viện, dependency, build pipeline, artifact hoặc thành phần bên thứ ba.
A04	Cryptographic Failures	Mã hóa, truyền dữ liệu, lưu secret hoặc bảo vệ dữ liệu nhạy cảm không đúng cách.
A05	Injection	Dữ liệu đầu vào bị đưa vào SQL, command, template hoặc interpreter mà không kiểm soát đúng.
A06	Insecure Design	Thiết kế nghiệp vụ hoặc kiến trúc thiếu cơ chế phòng vệ ngay từ đầu.
A07	Authentication Failures	Xác thực, token, session, reset password hoặc quản lý tài khoản có lỗi.
A08	Software or Data Integrity Failures	Không kiểm chứng tính toàn vẹn của dữ liệu, update, CI/CD hoặc code được nạp vào hệ thống.

Mã	Nhóm rủi ro	Ý nghĩa ngắn
A09	Security Logging and Alerting Failures	Thiếu log, monitoring hoặc cảnh báo khiến việc phát hiện sự cố bị chậm.
A10	Mishandling of Exceptional Conditions	Xử lý lỗi/ngoại lệ không an toàn, làm lộ thông tin hoặc tạo trạng thái hệ thống khó kiểm soát.

Lưu ý: OWASP Top 10 là nhóm rủi ro, không phải một checklist tự động hoàn chỉnh. Một tool có nhãn OWASP vẫn có thể bỏ sót lỗi thật hoặc báo lỗi cần kiểm chứng thủ công.

2. SAST và Semgrep

SAST (Static Application Security Testing) là kiểm thử bảo mật bằng cách phân tích mã nguồn hoặc file cấu hình mà không cần chạy ứng dụng. SAST phù hợp để phát hiện sớm hardcoded secret, pattern gọi API không an toàn, code injection, lỗi framework phổ biến và một số cấu hình rủi ro.

Semgrep là công cụ SAST chạy bằng CLI, dùng rule dạng pattern để tìm đoạn code có dấu hiệu nguy hiểm. Theo đề T09, flow chính dùng ruleset [p/owasp-top-ten](#) để quét EShop, sau đó output JSON được đưa vào script AI triage để sinh báo cáo kiểm chứng.

3. Cài đặt Semgrep

Chạy từ root repo [Seminar-SoftwareTesting](#).

3.1. Tạo môi trường Python cho script triage

```
python3 -m venv .venv
source .venv/bin/activate
```

Nếu dùng Windows PowerShell:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

3.2. Cài Semgrep CLI

Linux:

```
python3 -m pip install semgrep
semgrep --version
```

macOS:

```
brew install semgrep
semgrep --version
```

Windows PowerShell:

```
python -m pip install semgrep
semgrep --version
```

4. Flow Semgrep của nhóm

Flow đầy đủ bắt đầu từ quét mã nguồn bằng Semgrep, sau đó xuất kết quả JSON, kiểm tra finding thô, đưa vào AI triage, kiểm chứng lại bằng source context và cuối cùng ghi báo cáo.

4.1. Xác định source code cần quét

Chạy các lệnh từ root repo **Seminar-SoftwareTesting**. Theo README Semgrep hiện tại, mặc định nhóm đặt source EShop trong repo seminar tại **./eshop-sut**:

```
Seminar-SoftwareTesting/
├── eshop-sut/
└── src/semgrep/
```

Nếu source nằm đúng vị trí trên, đường dẫn scan là **./eshop-sut**. Nếu source nằm nơi khác, đặt biến **SOURCE_ROOT** bằng đường dẫn thật, ví dụ **/Users/mac/projects/eshop-sut**.

Trước khi quét, kiểm tra nhanh thư mục source:

```
ls ./eshop-sut
```

Không quét các thư mục dependency hoặc build output như `node_modules`, `dist`, `build`, `.next` vì chúng làm kết quả nhiễu và tốn thời gian.

4.2. Chọn source root khi source nằm nơi khác

Git Bash/Bash:

```
SOURCE_ROOT="/path/to/eshop-sut"
```

Windows PowerShell:

```
$env:SOURCE_ROOT="C:\path\to\eshop-sut"
```

4.3. Quét mã nguồn và xuất kết quả JSON

Trong flow chính, nhóm chạy Semgrep với `p/owasp-top-ten` theo đúng yêu cầu đề T09, đồng thời xuất kết quả ra JSON.

Nếu source nằm trong repo tại `./eshop-sut`:

Windows PowerShell:

```
New-Item -ItemType Directory -Force src/semgrep/output  
semgrep scan --config "p/owasp-top-ten" --exclude node_modules --exclude dist --  
exclude build --exclude .next --json -o src/semgrep/output/semgrep_results.json  
./eshop-sut
```

Git Bash/Bash:

```
mkdir -p src/semgrep/output  
semgrep scan \  
  --config "p/owasp-top-ten" \  
  --exclude node_modules \  
  --exclude dist \  
  ./eshop-sut
```

```
--exclude build \  
--exclude .next \  
--json \  
-o src/semgrep/output/semgrep_results.json \  
./eshop-sut
```

Nếu source đã có `.semgrepignore`, lệnh flow chính có thể rút gọn:

Windows PowerShell:

```
New-Item -ItemType Directory -Force src/semgrep/output  
semgrep scan --config "p/owasp-top-ten" --json -o  
src/semgrep/output/semgrep_results.json ./eshop-sut
```

Git Bash/Bash:

```
mkdir -p src/semgrep/output  
semgrep scan \  
  --config "p/owasp-top-ten" \  
  --json \  
  -o src/semgrep/output/semgrep_results.json \  
  ./eshop-sut
```

Nếu source nằm ở path khác:

Windows PowerShell:

```
semgrep scan --config "p/owasp-top-ten" --exclude node_modules --exclude dist --  
exclude build --exclude .next --json -o src/semgrep/output/semgrep_results.json  
$env:SOURCE_ROOT
```

Git Bash/Bash:

```
semgrep scan \  
  --config "p/owasp-top-ten" \  
  --exclude node_modules \  
  --exclude dist \  
  --exclude build \  
  --exclude .next \  
  --json \  
  -o src/semgrep/output/semgrep_results.json \  
  "$SOURCE_ROOT"
```

Ở bước này, đọc nhanh output trên terminal để nắm:

- File và dòng code có cảnh báo.
- Rule ID, severity và message của Semgrep.
- Đoạn code bị đánh dấu.
- Nhóm lỗi OWASP hoặc loại lỗi tương ứng, ví dụ secret hardcoded, HTTP không mã hóa, injection pattern.

Kiểm tra file kết quả đã được tạo:

```
ls src/semgrep/output/semgrep_results.json
```

File `semgrep_results.json` là bằng chứng scan gốc theo OWASP Top 10, nên giữ lại để đối chiếu với báo cáo triage và kết quả demo.

4.3.1. Quét mở rộng theo công nghệ EShop

Sau khi đã có kết quả bắt buộc theo `p/owasp-top-ten`, có thể chạy thêm ruleset theo công nghệ EShop để phân tích sâu hơn và phục vụ phần failure modes/rule coverage.

Windows PowerShell:

```
New-Item -ItemType Directory -Force src/semgrep/output
semgrep scan --config "p/owasp-top-ten" --config "p/nodejs" --config "p/javascript" -
--config "p/react" --config "src/semgrep/rules/eshop-security.yml" --exclude
node_modules --exclude dist --exclude build --exclude .next --json -o
src/semgrep/output/semgrep_results_extended.json ./eshop-sut
```

Git Bash/Bash:

```
mkdir -p src/semgrep/output
semgrep scan \
  --config "p/owasp-top-ten" \
  --config "p/nodejs" \
  --config "p/javascript" \
  --config "p/react" \
  --config "src/semgrep/rules/eshop-security.yml" \
  --exclude node_modules \
  --exclude dist \
  --exclude build \
  --exclude .next \
  --json \
  -o src/semgrep/output/semgrep_results_extended.json \
  ./eshop-sut
```

Lệnh mở rộng này không thay thế lệnh OWASP Top 10 trong flow chính; nó dùng để đổi chiều thêm khi cần phân tích vì sao ruleset mặc định có thể bỏ sót một số pattern trong EShop.

4.4. Cấu hình AI provider cho bước triage

Git Bash/Bash:

```
cp src/semgrep/.env.example src/semgrep/.env
```

Windows PowerShell:

```
copy src\semgrep\.env.example src\semgrep\.env
```

Điền key thật vào `src/semgrep/.env` nếu muốn gọi AI qua OpenRouter:

```
AI_PROVIDER=openai-compatible  
AI_MODEL=google/gemini-2.5-flash-lite  
AI_MAX_TOKENS=1800  
OPENROUTER_API_KEY=sk-or-v1-...  
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1
```

Nếu OpenRouter báo lỗi credit/token, giảm `AI_MAX_TOKENS`, ví dụ:

```
AI_MAX_TOKENS=1000
```

Không commit file `.env` thật vì có chứa API key.

4.5. Chạy AI triage trên kết quả Semgrep

Sau khi có `src/semgrep/output/semgrep_results.json`, chạy script triage. Nếu source nằm trong repo tại `./eshop-sut`:

Windows PowerShell:

```
python src/semgrep/semgrep_ai_triage.py src/semgrep/output/semgrep_results.json --  
source-root eshop-sut --output-dir src/semgrep/output
```

Git Bash/Bash:

```
python src/semgrep/semgrep_ai_triage.py \  
src/semgrep/output/semgrep_results.json \  
--source-root eshop-sut \  
--output-dir src/semgrep/output
```

Nếu source nằm ở path khác:

Windows PowerShell:

```
python src/semgrep/semgrep_ai_triage.py src/semgrep/output/semgrep_results.json --  
source-root $env:SOURCE_ROOT --output-dir src/semgrep/output
```

Git Bash/Bash:

```
python src/semgrep/semgrep_ai_triage.py \  
src/semgrep/output/semgrep_results.json \  
--source-root "$SOURCE_ROOT" \  
--output-dir src/semgrep/output
```

Script sẽ đọc từng finding trong JSON, lấy source context từ `--source-root`, tạo prompt phân tích và sinh báo cáo tổng hợp. AI được yêu cầu đọc source evidence trước, sau đó phân loại finding theo ba trạng thái:

- **True Positive**: source evidence khớp rule và lỗi có khả năng ảnh hưởng runtime/app thật.
- **False Positive**: source evidence hoặc vai trò file chứng minh finding không phải lỗi thật của ứng dụng.
- **Needs Human Review**: chưa đủ context về deploy, runtime reachability, config production hoặc dữ liệu nhạy cảm để chốt.

Nếu đang gọi AI thật và một finding bị lỗi provider/API, script sẽ dừng ngay với exit code **1**. Khi đó script không sinh report triage thiếu phân tích AI, để tránh nộp nhầm kết quả chưa đủ bằng chứng.

Nếu chưa có API key hoặc chỉ muốn tạo prompt/skeleton report để đọc thủ công, chạy offline:

Windows PowerShell:

```
python src/semgrep/semgrep_ai_triage.py src/semgrep/output/semgrep_results.json --  
source-root eshop-sut --output-dir src/semgrep/output --offline
```

Git Bash/Bash:

```
python src/semgrep/semgrep_ai_triage.py \  
src/semgrep/output/semgrep_results.json \  
--source-root eshop-sut \  
--output-dir src/semgrep/output \  
--offline
```

4.6. Kiểm tra output triage

Sau khi triage xong, kiểm tra các file chính:

Windows PowerShell:

```
Get-ChildItem src/semgrep/output  
Get-ChildItem src/semgrep/output/findings
```

Git Bash/Bash:

```
ls src/semgrep/output  
ls src/semgrep/output/findings
```

Output cần nộp hoặc dùng khi demo:

- `src/semgrep/output/semgrep_results.json`: kết quả scan gốc từ Semgrep.
- `src/semgrep/output/semgrep_triage_report.md`: báo cáo tổng hợp tất cả findings sau AI triage.
- `src/semgrep/output/semgrep_test_cases.md`: danh sách test case kiểm chứng từng finding, mỗi test case là một entry riêng.
- `src/semgrep/output/findings/`: prompt và output AI riêng cho từng finding.

Nội dung chính của `semgrep_triage_report.md`:

- Tổng quan số finding.
- Bảng tổng hợp bằng tiếng Việt: quy tắc, tệp, dòng, mức độ, CWE, OWASP, kết quả AI và trạng thái kiểm chứng.
- Chi tiết từng finding.
- Tags lỗi trong từng finding entry: Rule, Severity, CWE, OWASP, Likelihood, Impact, Confidence.
- Bảng chứng mã nguồn.
- Phân tích AI inline bằng tiếng Việt; heading do AI sinh ra được hạ cấp để không phá bố cục report.
- Checklist kiểm chứng thủ công.

Nội dung chính của `semgrep_test_cases.md`:

TC-SEMGREP-001

- Finding liên quan: SEMGREP-001
- Mục tiêu test: ...

Input

Request, headers và payload dùng để test.

Thao tác

Gửi request và ghi nhận status code, response body.

Kết quả cần ghi nhận

Nếu còn lỗi và nếu đã an toàn thì ghi nhận biểu hiện tương ứng.

Trạng thái

Chưa kiểm chứng

Output AI trong từng file `findings/*_ai_output.md` chỉ tập trung phân tích finding, không sinh test case chi tiết để tránh trùng với `semgrep_test_cases.md`. Mỗi output gồm:

1. Phân loại.
2. Lý do phân loại dựa trên source evidence.
3. Tác động thực tế trong bối cảnh EShop.
4. Cách khắc phục cụ thể.
5. Ghi chú cần tester kiểm tra thêm nếu chưa đủ context.

Khi đọc báo cáo, nhóm cần kiểm chứng lại từng finding bằng source code thật:

- Finding có nằm trong code tự viết của EShop hay chỉ nằm trong dependency/build output.
- Dữ liệu người dùng có đi tới sink nguy hiểm hay không.
- Secret, token, URL HTTP hoặc cấu hình yếu có dùng ở runtime hay chỉ là sample/test.
- AI có giải thích đúng file, dòng code, impact và remediation hay không.
- Kết luận cuối cùng là **True Positive**, **False Positive** hoặc **Needs Human Review**.

Trong output hiện có, Semgrep ghi nhận 12 findings, gồm hardcoded JWT secret trong `backend/server.js` và nhiều request HTTP không mã hóa trong `frontend-mobile/App.js`.

5. DAST và OWASP ZAP

DAST (Dynamic Application Security Testing) là kiểm thử bảo mật trên ứng dụng đang chạy thật. DAST không cần đọc source code; thay vào đó, tool gửi request HTTP, quan sát response, crawl trang, chạy passive scan và có thể active scan bằng payload kiểm thử.

OWASP ZAP là công cụ DAST mã nguồn mở. Trong project này, ZAP được dùng theo hai cách:

- Chạy bằng GUI để proxy trình duyệt, quan sát request/response, tạo context và cấu hình authentication thủ công.
- Chạy bằng CLI qua `src/zap/scan_zap.py` để tự động hóa scan, xuất report JSON/HTML và đưa JSON vào bước AI triage.

6. Cài đặt OWASP ZAP

Chạy từ root repo `Seminar-SoftwareTesting`.

6.1. Cài ZAP GUI

ZAP GUI cần Java 17 trở lên, trừ khi bản cài đặt đã bundle Java hoặc chạy bằng Docker.

Windows:

```
winget install --id=ZAP.ZAP -e
```

macOS:

```
brew install --cask zap
```

Linux qua Flatpak:

```
flatpak install flathub org.zaproxy.ZAP  
flatpak run org.zaproxy.ZAP
```

Linux qua Snap:

```
sudo snap install zaproxy --classic  
zaproxy
```

Qua trang chủ bằng package:

1. Mở <https://www.zaproxy.org/download/>.
2. Tải **Linux Package** nếu dùng Linux, hoặc **Cross Platform Package** nếu cần bản chạy đa nền tảng.
3. Giải nén file tải về, ví dụ:

```
tar -xf ZAP_<version>_Linux.tar.gz  
cd ZAP_<version>  
./zap.sh
```

Với Windows/macOS, trang Download cũng có installer riêng cho từng hệ điều hành (.exe cho Windows và .dmg cho macOS). Windows và Linux package cần Java 17+ đã cài sẵn; macOS installer đã bundle Java.

Các lỗi giao diện và môi trường thường gặp khi cài ZAP GUI (theo <src/zap/installation.md>):

- **Lỗi Headless Java (FATAL GuiBootstrap - ZAP GUI is not supported on a headless environment)**: Xảy ra khi Linux sử dụng Java headless làm mặc định.

Khắc phục bằng cách cài gói Java đầy đủ (có hỗ trợ đồ họa) và cấu hình lại Java mặc định:

```
sudo dnf install java-25-openjdk -y # Hoặc sudo apt install openjdk-17-jre
sudo alternatives --config java      # Chọn phiên bản không chứa chữ 'headless'
```

- **Lỗi hiển thị cửa sổ trắng / không tương tác được trên Wayland (Linux):** Ứng dụng Java AWT/Swing bị lỗi khi chạy native trên Wayland. Ép ZAP chạy qua XWayland bằng lệnh:

```
_JAVA_AWT_WM_NONREParenting=1 GDK_BACKEND=x11 ./zap.sh
```

- **Cảnh báo bảo mật khi tải và mở ứng dụng:**
 - Windows: Trên trình duyệt click ... -> Chọn **Keep** -> **Show more** -> **Keep anyway**.
 - macOS: Nếu báo "ZAP.app cannot be opened", vào **System Preferences** > **Security & Privacy** > bấm **Open Anyway**.

6.2. Cài dependency cho CLI flow

Tạo hoặc kích hoạt virtual environment:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install python-owasp-zap-v2.4
```

Nếu dùng Windows PowerShell:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install python-owasp-zap-v2.4
```

CLI flow cần ZAP daemon tại <http://localhost:8090>. Có thể chạy bằng Docker:

```
docker run --rm -d --name eshop-zap --network host \
-v $(pwd):$(pwd) \
ghcr.io/zaproxy/zaproxy:stable zap.sh -daemon \
-port 8090 -host 0.0.0.0 -config api.disablekey=true
```

6.3. Chuẩn bị cấu hình ZAP

```
cp src/zap/.env.example src/zap/.env
```

Các biến quan trọng trong `src/zap/.env`:

```
ZAP_TARGET=http://localhost:3000
ZAP_URL=http://localhost:8090
ZAP_AUTH_ROLE=none
ZAP_MAX_URLS=300
ZAP_REPORT_FORMAT=json
ZAP_REPORT_FILE=src/zap/output/backend_basic.json
```

`ZAP_MAX_URLS` là giới hạn số URL trước khi chạy active scan. Nếu spider/AJAX Spider crawl vượt giới hạn này, script vẫn giữ kết quả spider/passive scan và xuất report, nhưng bỏ qua active scan để tránh quá tải RAM. Có thể bỏ trống biến này nếu muốn active scan không bị giới hạn.

Nếu scan có authentication, điền credential test tương ứng:

```
ZAP_USER_EMAIL=test@eshop.com
ZAP_USER_PASSWORD=Test1234!
ZAP_ADMIN_EMAIL=admin@eshop.com
ZAP_ADMIN_PASSWORD=Admin123!
```

Không commit file `.env` thật vì có thể chứa credential hoặc API key.

7. Chạy ZAP bằng GUI

Chỉ scan hệ thống local/lab mà nhóm có quyền kiểm thử. Trước khi chạy GUI, khởi động EShop:

- Backend: `http://localhost:3000`

- Frontend Web/User: <http://localhost:5173>
- Frontend Admin: <http://localhost:5174>

7.1. GUI scan không login

Flow này dùng để scan phần public như trang sản phẩm, endpoint public hoặc backend API không cần token.

1. Mở OWASP ZAP GUI.
2. Ở màn hình session, chọn không lưu session nếu demo nhanh, hoặc lưu session nếu cần evidence.
3. Vào **Quick Start** -> **Automated Scan**.
4. Nhập target public, ví dụ <http://localhost:3000> hoặc <http://localhost:5173> hoặc <http://localhost:5174>.
5. Nhấn **Attack**.
6. Sau khi scan xong, xem:
 - **Sites**: URL đã crawl được.
 - **History**: request/response runtime.
 - **Alerts**: danh sách cảnh báo.
 - **Request/Response**: evidence của từng alert.

Với frontend SPA, nên dùng thêm **Manual Explore** hoặc **AJAX Spider** vì nhiều route/API chỉ xuất hiện sau khi JavaScript chạy.

7.2. Cấu hình browser proxy cho GUI

Để ZAP bắt được traffic thực từ người dùng, cấu hình Firefox đi qua ZAP proxy:

1. Trong ZAP, vào **Tools** -> **Options...** -> **Network** -> **Local Servers/Proxies**.
2. Xác nhận main proxy là localhost:8080.
3. Trong Firefox, vào **Settings** -> **Network Settings**.
4. Chọn manual proxy và nhập HTTP proxy localhost, port [8080](http://localhost:8080).
5. Nếu traffic localhost không xuất hiện trong ZAP, mở <about:config> và đặt [network.proxy.allow_hijacking_localhost](http://localhost)=true.

Hình minh họa chi tiết nằm trong src/zap/gui_scan.md.

7.3. GUI scan có authentication

Flow này dùng để scan các chức năng cần đăng nhập như giỏ hàng, profile hoặc API user/admin.

1. Mở EShop trong Firefox đã cấu hình proxy.
2. Đăng nhập hoặc đăng ký bằng tài khoản test để ZAP ghi nhận request login.
3. Trong cây **Sites**, đưa cả frontend **http://localhost:5173** và backend **http://localhost:3000** vào cùng context.
4. Chọn request **POST /api/login**, click chuột phải và chọn **Flag as Context** -> context tương ứng -> **JSON-based Auth Login Request**.
5. Trong context, mở phần **Authentication** và kiểm tra login URL, method, payload JSON và dấu hiệu đăng nhập thành công.
6. Mở tab **Users**, tạo user test và nhập username/password.
7. Bật **Forced User Mode** để ZAP gán user đã cấu hình vào request trong context.
8. Chạy **Spider...** hoặc **AJAX Spider...** trên context. Với frontend React/SPA, ưu tiên bật AJAX Spider.
9. Chỉ chạy **Active Scan...** sau khi scope đúng, số URL trong **Sites** không quá lớn, và tài khoản test có thể bị thay đổi dữ liệu mà không ảnh hưởng môi trường thật. Nếu AJAX Spider tạo hàng trăm hoặc hàng nghìn URL, nên dừng ở passive scan cho frontend rồi active scan backend/API riêng.

Kết quả cần lưu lại:

- Host frontend/backend trong **Sites**.
- Request login và request xác minh user như **/api/users/me**.
- Alert trong tab **Alerts**.
- Request/response evidence cho từng alert quan trọng.

8. Chạy ZAP bằng CLI trong **src/zap**

Script **src/zap/scan_zap.py** tự động kết nối ZAP daemon, chuẩn bị context/auth nếu cần, chạy spider, tùy chọn AJAX Spider, passive scan, active scan và export report.

Script hiện hỗ trợ target local trên các cổng **3000**, **5173**, **5174**. Với frontend SPA, dùng thêm **--max-urls** để đặt ngân sách URL trước active scan; nếu crawl vượt ngân sách, script sẽ bỏ qua active scan để tránh tràn RAM.

8.1. Chạy CLI không login

Dùng khi scan backend public hoặc frontend public, không cần token/cookie.

Backend API, xuất JSON để đưa vào AI:

```
python src/zap/scan_zap.py \  
  --target http://localhost:3000 \  
  --report-format json \  
  --output-file src/zap/output/backend_basic.json
```

Frontend user public, có AJAX Spider để khám phá route SPA:

```
python src/zap/scan_zap.py \  
  --target http://localhost:5173 \  
  --ajax-spider \  
  --report-format json \  
  --output-file src/zap/output/frontend_public_basic.json
```

Nếu chỉ cần đọc thủ công, đổi `--report-format html` và đặt output `.html`.

8.2. Chạy CLI có authentication

Dùng khi cần scan dưới quyền user hoặc admin. Trước khi chạy, đảm bảo `src/zap/.env` có credential đúng và backend đang chạy.

Frontend user:

```
python src/zap/scan_zap.py \  
  --target http://localhost:5173 \  
  --auth-role user \  
  --forced-user \  
  --ajax-spider \  
  --report-format json \  
  --output-file src/zap/output/frontend_user_basic.json
```

Frontend admin:

```
python src/zap/scan_zap.py \  
  --target http://localhost:5174 \  
  --auth-role admin \  
  --forced-user \  
  --ajax-spider \  
  --report-format json \  
  --output-file src/zap/output/frontend_admin_basic.json
```

Backend authenticated API scan dưới quyền user:

```
python src/zap/scan_zap.py \
  --target http://localhost:3000 \
  --auth-role user \
  --forced-user \
  --report-format json \
  --output-file src/zap/output/backend_user_basic.json
```

Nếu scan authenticated báo 401/403, kiểm tra credential trong `.env`, endpoint login `http://localhost:3000/api/login`, và endpoint verify `http://localhost:3000/api/users/me`.

Khi thấy log dạng `URL budget: discovered 13055/300 URLs for active scan`, nghĩa là frontend đã crawl vượt `--max-urls`; lúc đó report vẫn có dữ liệu spider/passive scan, nhưng sẽ không có bằng chứng active scan cho target đó. Nếu cần active scan, ưu tiên chạy trên backend/API `http://localhost:3000` hoặc giảm scope frontend trước khi tăng giới hạn.

8.3. Ý nghĩa flag chính

Flag	Ý nghĩa
<code>--target</code>	URL cần scan: backend 3000, frontend user 5173, frontend admin 5174.
<code>--zap-url</code>	URL ZAP daemon, mặc định <code>http://localhost:8090</code> .
<code>--api-key</code>	API key nếu ZAP daemon bật key. Lab mặc định dùng <code>api.disablekey=true</code> .
<code>--auth-role</code>	Chọn tài khoản seed để đăng nhập: <code>none</code> , <code>user</code> , <code>admin</code> .
<code>--forced-user</code>	Bật Forced User Mode để scan dưới user đã cấu hình.
<code>--ajax-spider</code>	Dùng AJAX Spider, cần thiết cho frontend React/SPA.
<code>--max-urls</code>	Giới hạn số URL trước active scan; nếu vượt giới hạn thì bỏ qua active scan để tránh quá tải RAM.
<code>--scan-mode</code>	Mode scan: <code>default</code> (mặc định) hoặc <code>owasp-top10-2025</code> (chỉ bật scanners thuộc OWASP Top 10 2025).
<code>--report-format</code>	<code>html</code> để đọc thủ công, <code>json</code> để pipeline/AI xử lý.

Flag	Ý nghĩa
<code>--output-file</code>	Đường dẫn file report đầu ra.

8.4. Kiểm tra output scan

Sau khi chạy CLI, kiểm tra file report:

```
ls src/zap/output
```

Output chính thường dùng:

- `src/zap/output/backend_basic.json`: report backend public.
- `src/zap/output/frontend_user_basic.json`: report frontend dưới quyền user.
- `src/zap/output/frontend_admin_basic.json`: report frontend dưới quyền admin.

Khi đọc report, nhóm cần kiểm chứng:

- Alert thuộc đúng target và scope EShop.
- Endpoint có public hay cần authentication.
- Evidence trong request/response có đủ để kết luận lỗi thật.
- Alert là lỗi runtime thật hay noise do môi trường local HTTP/dev server.

9. Sử dụng AI để phân tích ZAP report

Script `src/zap/zap_ai_triage.py` đọc một hoặc nhiều report JSON của ZAP, chuẩn hóa endpoint/evidence runtime và sinh báo cáo Markdown. Flow ZAP được tách riêng với Semgrep: `zap_triage_report.md` gom theo alert group và liệt kê endpoint trong từng alert, còn `zap_test_cases.md` vẫn tạo test case cho từng endpoint/request instance. Script ưu tiên OpenAI nếu có `OPENAI_API_KEY`, nếu không thì dùng OpenRouter khi có `OPENROUTER_API_KEY`. Nếu thiếu cả hai key, script báo lỗi cấu hình AI; dùng `--offline` nếu chỉ muốn tạo prompt/report skeleton mà chưa gọi AI.

9.1. Cấu hình AI provider

Mở `src/zap/.env` và điền một trong hai nhóm cấu hình.

OpenAI:

```
OPENAI_API_KEY=your_openai_key
OPENAI_MODEL=gpt-4.1-mini
OPENAI_BASE_URL=https://api.openai.com/v1
```

OpenRouter:

```
OPENROUTER_API_KEY=your_openrouter_key
OPENROUTER_MODEL=google/gemini-2.5-flash
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1
OPENROUTER_MAX_TOKENS=4096
```

Không commit API key thật vào repository.

9.2. Chạy AI triage cho một hoặc nhiều report

```
python src/zap/zap_ai_triage.py \
  src/zap/output/backend_basic.json \
  src/zap/output/frontend_user_basic.json \
  src/zap/output/frontend_admin_basic.json \
  --output-dir src/zap/output/markdown \
  --target-prefix http://localhost:3000 \
  --target-prefix http://localhost:5173 \
  --target-prefix http://localhost:5174
```

Ý nghĩa các argument/flag trong lệnh:

Argument/flag	Ý nghĩa
<code>src/zap/output/backend_basic.json</code> <code>src/zap/output/frontend_user_basic.json</code> <code>src/zap/output/frontend_admin_basic.json</code>	Danh sách một hoặc nhiều file JSON report của ZAP. Đây là positional arguments, không dùng <code>--input</code> .
<code>--output-dir src/zap/output/markdown</code>	Thư mục output cho Markdown report. Script sẽ tạo <code>zap_triage_report.md</code> , <code>zap_test_cases.md</code> và thư mục <code>alerts/</code> bên trong thư mục này.

Argument/flag	Ý nghĩa
<code>--target-prefix http://localhost:3000</code>	Chỉ giữ các alert instance có URL bắt đầu bằng backend target.
<code>--target-prefix http://localhost:5173</code>	Chỉ giữ các alert instance thuộc frontend user target. Flag này có thể truyền nhiều lần.
<code>--target-prefix http://localhost:5174</code>	Chỉ giữ các alert instance thuộc frontend admin target. Nếu không truyền <code>--target-prefix</code> , script sẽ lấy tất cả URL xuất hiện trong JSON ZAP, kể cả URL ngoài scope nếu ZAP ghi nhận được.

Nếu chỉ có một report:

```
python src/zap/zap_ai_triage.py \
  src/zap/output/backend_basic.json \
  --output-dir src/zap/output/markdown_backend \
  --target-prefix http://localhost:3000
```

Nếu chưa có API key hoặc chỉ muốn kiểm tra format Markdown/prompt trước khi gọi AI, chạy offline:

```
python src/zap/zap_ai_triage.py \
  src/zap/output/backend_basic.json \
  src/zap/output/frontend_user_basic.json \
  src/zap/output/frontend_admin_basic.json \
  --output-dir src/zap/output/markdown \
  --target-prefix http://localhost:3000 \
  --target-prefix http://localhost:5173 \
  --target-prefix http://localhost:5174 \
  --offline
```

Các flag tùy chọn khác:

Flag	Ý nghĩa
-- offline	Không gọi OpenAI/OpenRouter. Script chỉ sinh prompt, skeleton AI output và Markdown testcase.
--limit N	Giới hạn số alert instance được đọc trước khi gom nhóm. Chỉ nên dùng để debug nhanh, không dùng cho report nộp chính thức.

Sau khi chạy thành công, kiểm tra các output chính:

- `src/zap/output/markdown/zap_triage_report.md`: báo cáo triage theo alert group, mỗi group có danh sách endpoint bị ảnh hưởng và runtime evidence đại diện.
- `src/zap/output/markdown/zap_test_cases.md`: toàn bộ test case theo từng endpoint/request instance.
- `src/zap/output/markdown/alerts/*_prompt.md`: prompt tiếng Việt gửi AI cho từng alert group.
- `src/zap/output/markdown/alerts/*_ai_output.md`: output AI tương ứng từng alert group.

10. Ví dụ testcase từ Semgrep và ZAP

Sau khi chạy AI triage, mỗi nhánh Semgrep và ZAP đều sinh ra test case riêng để người kiểm thử tái hiện lại finding/alert bằng evidence tương ứng.

10.1. Ví dụ testcase Semgrep

Ví dụ dưới đây tương ứng với finding hardcoded JWT secret của Semgrep:

- Rule ID: `javascript.jsonwebtoken.security.jwt-hardcode.hardcoded-jwt-secret`
- File: `backend/server.js`
- CWE: CWE-798
- OWASP: A07 Authentication Failures

Testcase mẫu:

TC-SEMGREP-001

- Finding liên quan: SEMGREP-001

- Mục tiêu test: Kiểm chứng khả năng giả mạo JWT khi **SECRET_KEY** bị hardcode trong source code.

Input

- Source evidence:
 - `const SECRET_KEY = "super_secret_key_that_should_not_be_here";`
 - `jwt.sign(..., SECRET_KEY)`
 - `jwt.verify(..., SECRET_KEY)`
- PoC: `src/semgrep/exploit.js`

Thao tác

1. Chạy PoC để tạo forged token:

```
node src/semgrep/exploit.js
```

2. Copy token sinh ra từ PoC.

3. Gửi request kiểm tra:

```
curl -i http://localhost:3000/api/users/me \
-H "Authorization: Bearer <forged_token>"
```

4. Ghi nhận status code, response body và quyền mà backend suy ra từ token.

Kết quả cần ghi nhận

- Nếu còn lỗi: backend chấp nhận forged token, cho phép truy cập như user/admin hợp lệ.
- Nếu đã an toàn: forged token bị từ chối, backend trả **401/403** hoặc lỗi verify JWT.

Trạng thái

Chưa kiểm chứng

Phần AI triage liên quan:

Trích AI triage cho SEMGREP-001

1. Giải thích lỗ hổng

Lỗi hỏng xảy ra do khóa bí mật (`SECRET_KEY`) dùng để ký và xác thực JWT được khai báo tĩnh trực tiếp trong mã nguồn tại `backend/server.js`.

2. Proof of Concept

Khi có được chuỗi `super_secret_key_that_should_not_be_here`, kẻ tấn công có thể tự tạo một JWT hợp lệ:

```
const jwt = require('jsonwebtoken');
const LEAKED_SECRET = "super_secret_key_that_should_not_be_here";
const payload = { id: 1, role: "admin" };
const forgedToken = jwt.sign(payload, LEAKED_SECRET);
console.log("Bearer " + forgedToken);
```

3. Mức độ ảnh hưởng

- Authentication Bypass.
- Privilege Escalation.
- Truy cập, sửa đổi hoặc xóa dữ liệu trái phép nếu token giả được backend tin tưởng.

4. Khuyến nghị khắc phục

Không lưu secret trong source code. Chuyển `SECRET_KEY` sang biến môi trường như `process.env.JWT_SECRET`, không commit `.env`, và rotate toàn bộ token cũ sau khi thay secret.

Ý nghĩa của testcase này là: AI triage đã chỉ ra root cause nằm ở secret hardcoded và mô tả rõ đường khai thác bằng forged JWT; vì vậy testcase tập trung vào việc chứng minh forged token có được backend chấp nhận hay không.

10.2. Ví dụ testcase ZAP

Ví dụ dưới đây lấy từ testcase đầu tiên trong `src/zap/output/markdown/zap_test_cases.md`, tương ứng với alert `ZAP-001`.

Testcase mẫu:

TC-ZAP-001

- Alert liên quan: ZAP-001

- Mục tiêu test: Kiểm chứng alert **CSP: Failure to Define Directive with No Fallback** bằng cách replay request runtime mà ZAP đã ghi nhận.
- Source JSON: **backend_basic.json**

Input

```
GET http://localhost:3000
```

Headers:

```
GET http://localhost:3000 HTTP/1.1
host: localhost:3000
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
pragma: no-cache
cache-control: no-cache
Authorization: Bearer <token_da_zap_gghi_nhan>
```

Payload:

```
{}
```

Thao tác

1. Đảm bảo backend/frontend EShop và ngữ cảnh auth tương ứng đang chạy.
2. Chuẩn bị token, cookie hoặc session nếu request của ZAP có header xác thực.
3. Replay request theo method, URL, headers từ ZAP.
4. Ghi nhận status code, response headers và response body.
5. So sánh kết quả mới với evidence ZAP trong triage report.

Kết quả cần ghi nhận

- Nếu còn lỗi: Response vẫn chứa evidence **default-src 'none'** hoặc hành vi runtime vẫn khớp alert **CSP: Failure to Define Directive with No Fallback**.
- Nếu đã an toàn: Response không còn evidence liên quan hoặc server trả trạng thái chặn/hardening phù hợp.

Trạng thái

Phần AI triage liên quan:

Trích AI triage cho ZAP-001

ZAP-001: CSP: Failure to Define Directive with No Fallback

- Phân loại AI: True Positive
- Risk: Medium
- Confidence: High
- Evidence đại diện: `Content-Security-Policy: default-src 'none'`

Lý do phân loại

- Các response quan sát được đều trả về header `Content-Security-Policy` với giá trị `default-src 'none'`.
- AI triage đánh giá policy CSP hiện tại chưa định nghĩa đầy đủ các directive cần thiết, nên đây là vấn đề cấu hình có thật ở cấp hệ thống.

Tác động thực tế

- CSP cấu hình chưa đầy đủ có thể làm giảm hiệu quả phòng vệ trước XSS.
- Vấn đề ảnh hưởng rộng vì cùng một pattern xuất hiện trên nhiều endpoint/response.

Khuyến nghị khắc phục

- Bổ sung đầy đủ các directive như `script-src`, `style-src`, `img-src`, `connect-src`.
- Kiểm tra lại CSP ở các response chính sau khi sửa để xác nhận policy đã chặt chẽ và nhất quán.

Ý nghĩa của testcase này là: AI triage đã xác định đây là **True Positive** dựa trên runtime evidence mà ZAP thu được, nên testcase tập trung vào việc replay lại chính request đó để kiểm tra response hiện tại còn chứa CSP evidence tương tự hay không.

11. Ranh giới trách nhiệm: Tester vs Developer vs SOC

Khi dùng Semgrep, ZAP và AI triage, người thực hiện kiểm thử cần phân biệt rõ trách nhiệm của từng vai trò khi một finding được phát hiện. Alert từ công cụ chưa tự động đồng nghĩa với lỗi đã xác nhận; mỗi vai trò cần xử lý đúng phần việc của mình để kết luận cuối cùng có bằng chứng.

Vai trò	Trách nhiệm chính	Output cần bàn giao
Tester / Security Tester	Chạy Semgrep/ZAP, triage alert, tái hiện lỗi bằng PoC, phân loại True Positive , False Positive hoặc Needs Human Review .	Report có bước tái hiện, evidence source/runtime, mức độ ảnh hưởng và điều kiện xảy ra lỗi.
Developer	Đọc report, xác định nguyên nhân trong code/config, sửa lỗi, bổ sung test chống tái diễn và phản hồi nếu finding là false positive hoặc chỉ xảy ra trong môi trường dev.	Patch, test case, giải thích kỹ thuật và trạng thái sau khi verify lại.
SOC / Security Operation Center	Theo dõi hệ thống thật sau deploy, phát hiện dấu hiệu khai thác qua log/alert, phối hợp xử lý incident nếu lỗi đã ảnh hưởng production.	Alert vận hành, timeline sự cố, mức độ ảnh hưởng thực tế và khuyến nghị phản ứng.

Tóm lại, tester chịu trách nhiệm tìm và chứng minh lỗi, developer chịu trách nhiệm sửa lỗi trong sản phẩm, còn SOC chịu trách nhiệm giám sát và phản ứng khi rủi ro xuất hiện trên môi trường vận hành. Trong seminar này, phần demo/report cần thể hiện ranh giới đó để tránh nhầm lẫn giữa phát hiện tự động, lỗi đã được kiểm chứng và sự cố thật ngoài production.

12. Xử lý sự cố

Vấn đề	Dấu hiệu	Nguyên nhân thường gặp	Cách xử lý
Không cài được package Python global	externally-managed-environment	Distro chặn cài pip vào system Python	Dùng .venv rồi cài dependency trong virtual environment.
Semgrep không tìm	Lệnh scan báo không có	Source EShop nằm	Kiểm tra ls ./eshop-sut ; nếu source nằm nơi khác, đặt lại SOURCE_ROOT bằng

Vấn đề	Dấu hiệu	Nguyên nhân thường gặp	Cách xử lý
thấy source	<code>./eshop-sut</code> hoặc <code>\$SOURCE_ROOT</code>	khác vị trí mặc định hoặc biến <code>SOURCE_ROOT</code> sai	đường dẫn source thật.
OpenRouter trả 402	AI triage dừng với exit code 1	Hết credit hoặc model/request quá tốn token	Giảm <code>AI_MAX_TOKENS</code> , dùng model nhẹ hơn, nạp credit hoặc chạy <code>--offline</code> để tạo prompt/skeleton đọc thủ công.
ZAP không khởi động	Docker unavailable hoặc image pull lỗi	Docker chưa chạy, thiếu quyền, mạng chậm	Kiểm tra Docker daemon, pull trước image ZAP hoặc dùng ZAP GUI/daemon tự chạy tại <code>localhost:8090</code> .
ZAP GUI bị crash headless	<code>FATAL GuiBootstrap - ZAP GUI is not supported on a headless environment</code>	Môi trường Linux dùng Java headless làm mặc định	Cài package Java GUI (<code>java-25-openjdk</code> hoặc <code>openjdk-17-jre</code>), gõ <code>sudo alternatives --config java</code> chọn bản non-headless.
ZAP GUI trắng xóa / đơ	Cửa sổ ZAP mở lên trắng xóa hoặc không click được	Lỗi ứng dụng Java AWT/Swing khi chạy native trên Wayland	Chạy ZAP bằng lệnh: <code>_JAVA_AWT_WM_NONREParenting=1 GDK_BACKEND=x11 ./zap.sh</code> .
Authenticated ZAP scan lỗi 401/403	Login hoặc <code>/api/users/me</code> fail	Sai credential, account bị khóa, backend chưa chạy	Reset dữ liệu test, đổi credential trong <code>.env</code> , kiểm tra backend 3000.

Vấn đề	Dấu hiệu	Nguyên nhân thường gặp	Cách xử lý
Report backend lẫn frontend	JSON/HTML có nhiều site khác target	ZAP daemon giữ session cũ hoặc output file đặt nhầm	Tạo session mới/clear history trong ZAP, hoặc dùng file output riêng cho từng target.

13. Tài liệu tham khảo

- OWASP Top 10 2025: <https://owasp.org/www-project-top-ten/>
- Semgrep documentation: <https://semgrep.dev/docs/>
- OWASP ZAP Getting Started: <https://www.zaproxy.org/getting-started/>
- OWASP ZAP Download: <https://www.zaproxy.org/download/>
- OWASP ZAP Report Generation:
<https://www.zaproxy.org/docs/desktop/addons/report-generation/>
- Tài liệu trong repo: [docs/semgrep/](#), [src/semgrep/](#), [docs/zap/](#), [src/zap/](#), [weekly-reports/Group06_06/Group06.md](#).

14. Công bố sử dụng AI

Nhóm có sử dụng AI để hỗ trợ soạn thảo hướng dẫn, phân tích Semgrep findings và triage ZAP alerts. Các command, flag CLI, đường dẫn output và nhận định kỹ thuật cần được thành viên nhóm kiểm tra lại bằng source code, report JSON/HTML và request runtime trước khi đưa vào kết luận cuối cùng.